Learn Database Normalization

Detailed Notes

What Is Normalization?

Normalization is the process of structuring relational database tables so that they cannot express logically contradictory or redundant information. A normalized table can only contain *one version of the truth.*

Motivation

- Prevent impossible situations (e.g., one customer with two dates of birth).
- Preserve data integrity: data should not "disagree with itself."
- Make tables easier to understand, extend, and query.
- Avoid anomalies:
  - Insertion anomalies
  - Update anomalies
  - Deletion anomalies

Think of normal forms like safety levels for a bridge:

- "Passes 1NF" $\approx$ Basic safety.
- "Passes 2NF, 3NF..." $\approx$ Progressively stronger safety guarantees.

First Normal Form (1NF)

1NF is about the **shape and atomicity** of your table and the fact that rows in a relational table have no inherent order.

A table is in 1NF if it satisfies *all* of the following:

1. No Row-Order Semantics

- You cannot use row order to encode meaning.
- **Example:** A list of Beatles "from tallest to shortest" stored by ordering rows. In a relational table the DB is free to return rows in any order, so "tallest to shortest" cannot be encoded by row order alone.
- **Fix:** Make ordering explicit with a column (e.g., `Height_cm` or `Rank`), not implicit in row order.

2. Columns Have a Single, Consistent Data Type

- No mixing of types in the same column (e.g., integers and strings together).
- **Example:** A `Height_In_Cm` column must be defined as integer, and all values must be integers.
- Spreadsheets often allow mixing types; relational DBs do not.

3. Every Table Has a Primary Key

- A primary key (PK) uniquely identifies each row.
- Without a PK, multiple identical or contradicting rows can appear.
- **Example:** `Beatle_Height(Beatle, Height_In_Cm)`. PK = `Beatle`. The DB will prevent multiple rows for the same Beatle.

4. No Repeating Groups

- You cannot store a whole "group" of related items in a single row/column.
- **Example:** Game inventory per player with many item types.
  - **Bad designs:**
    - A single text column: `Inventory = '5 arrows, 3 shields, 10 copper coins'`.
    - A single row with many columns: `Item1_Type, Item1_Qty, Item2_Type, Item2_Qty, ...`
  - **Problems:**
    - Queries like "who has > 10 copper coins?" become ugly or impossible to do efficiently.
    - Super-wide tables do not scale as item types grow.
- **Good 1NF design:**
  `Player_Inventory(Player, Item_Type, Item_Quantity)`
  PK = `(Player, Item_Type)`. Each row = one player–item pair with a quantity.

Summary of 1NF Rules

1. Using row order to convey information is not permitted.
2. Mixing data types in the same column is not permitted.
3. Having a table without a primary key is not permitted.
4. Repeating groups are not permitted.

Second Normal Form (2NF)

2NF concerns **composite primary keys** and how non-key attributes depend on them.

Setup

Start from the 1NF-compliant `Player_Inventory` table:

- Columns: `Player`, `Item_Type`, `Item_Quantity`.
- PK = `(Player, Item_Type)`.
- `Item_Quantity` is a non-key attribute (not part of the PK).
- Table is in 1NF.

Enhancement: Add `Player_Rating` (Beginner / Intermediate / Advanced) as a column in `Player_Inventory`.

Anomalies That Arise

1. **Deletion anomaly.** If player `gila19` has zero items (inventory now empty), her only row in `Player_Inventory` is deleted. We lose her `Player_Rating` entirely, even though her rating still conceptually exists.
2. **Update anomaly.** If player `jdog21` improves from Intermediate to Advanced, we must update *all* rows for `jdog21`. If we accidentally update only some rows, the DB shows conflicting ratings for `jdog21` at once.
3. **Insertion anomaly.** New player `tina42` (rating Beginner) with no items yet. We cannot insert a row for her in `Player_Inventory` without inventing items; thus we cannot record her rating.

Reason: Violation of 2NF

- Non-key attributes: `Item_Quantity`, `Player_Rating`.
- PK: `(Player, Item_Type)`.
- Dependencies:
  - `Item_Quantity` depends on the pair `(Player, Item_Type)` → OK (depends on the *whole* key).
  - `Player_Rating` depends only on `Player` (a part of the key) → **part-key dependency** → not OK.
- **Informal definition of 2NF:** Each non-key attribute must depend on the *entire* primary key, not just part of it.

Fix (How to Reach 2NF)

- Recognize `Player` as an entity in its own right.
- Create a separate `Player` table:
  `Player(Player_ID, Player_Rating, Date_Of_Birth, Email, ...)`
- Keep `Player_Inventory(Player, Item_Type, Item_Quantity)` as before.
- Now:
  - In `Player_Inventory`, every attribute (`Item_Quantity`) depends on the whole key `(Player, Item_Type)`.
  - In `Player`, every attribute depends on `Player_ID` only.
- Both tables satisfy 2NF.

Third Normal Form (3NF) and Boyce–Codd Normal Form (BCNF)

3NF deals with **transitive dependencies** between non-key attributes.

Setup

Start from the `Player` table (already in 2NF):

- `Player(Player_ID, Player_Rating, Player_Skill_Level, ...)`
- `Player_Skill_Level`: integer scale 1–9.
- Mapping:
  - Beginner: skill 1–3
  - Intermediate: skill 4–6
  - Advanced: skill 7–9

Potential Inconsistency

- Suppose `gila19`'s skill level increases from 3 to 4.
- We update `Player_Skill_Level` to 4 but forget to update `Player_Rating` from Beginner to Intermediate.
- Table now says:
  - `Player_Skill_Level = 4` ⇒ she should be Intermediate.
  - `Player_Rating = Beginner`.
- Data is logically inconsistent.

Why 2NF Did Not Catch This

- PK is a single attribute (`Player_ID`), so there are no part-key dependencies.
- Both `Player_Rating` and `Player_Skill_Level` depend on the key, but in different ways:
  - `Player_ID` → `Player_Skill_Level` (direct).
  - `Player_Skill_Level` → `Player_Rating` (rating is derived from skill).
  - So `Player_ID` → `Player_Rating` holds *transitively* via `Player_Skill_Level`.

Transitive Dependency

- A non-key attribute depends on another non-key attribute, which depends on the key.
- 3NF forbids this: non-key attributes must not depend on other non-key attributes.

Fix (To Reach 3NF)

- Remove `Player_Rating` from the `Player` table.
- Keep `Player(Player_ID, Player_Skill_Level, ...)`.
- Create a `Player_Skill_Levels` table describing the mapping: e.g., rows like `(Skill_Level, Min, Max, Player_Rating)` or a simpler form that maps ranges to ratings.
- Now `Player_Rating` is derived when needed by joining with `Player_Skill_Levels`.
- No non-key attribute depends on another non-key attribute inside `Player`.

Key Slogan for 3NF

> *Every non-key attribute in a table should depend on the key, the whole key, and nothing but the key.*

BCNF (Boyce–Codd Normal Form)

- Stronger version of 3NF.
- Informal guideline: *every* attribute (not just non-key) must depend on the key, the whole key, and nothing but the key.
- In practice:
  - Most well-designed 3NF tables are also in BCNF.
  - The difference only shows up in more obscure cases with multiple overlapping candidate keys.

Fourth Normal Form (4NF)

4NF is about **multivalued dependencies** (MVDs).

Scenario: DesignMyBirdhouse.com

Customers choose:

- Birdhouse model (e.g., Prairie, Cottage, etc.).
- Color (e.g., brown, beige, green).
- Style (e.g., bungalow, schoolhouse).

Each model has its own set of allowed colors and styles.

Naïve Table

- `Model_Colors_And_Styles_Available(Model, Color, Style)`
- PK = `{Model, Color, Style}` (all 3 columns).
- The table is in 3NF: every attribute is part of the key; no transitive dependencies.

Anomalies Despite 3NF

- Suppose Prairie supports colors {brown, beige}, styles {bungalow, schoolhouse}.
- We decide to add `green` as a new color for Prairie.
- Correct data requires *two* new rows:
  - Prairie–green–bungalow
  - Prairie–green–schoolhouse
- If we add only one of these rows (e.g., bungalow only), the data incorrectly suggests Prairie–green–schoolhouse is not available, even though the business rules say all styles should be available in all colors for that model.

Core Logical Structure

Model determines:

- A set of colors.
- A set of styles.

Colors and styles are *independent* choices for a given model.

Multivalued Dependency (MVD)

- We write: `Model` $\twoheadrightarrow$ `Color` (each model has a set of possible colors).
- And `Model` $\twoheadrightarrow$ `Style` similarly.
- An MVD (double arrow) means: "for a given left-hand side value, there is a set of right-hand side values, independent of other columns."

4NF Rule (Informal)

The only multivalued dependencies allowed in a table are those where the determinant is a key. In the example, `Model` is not the whole key (`{Model, Color, Style}` is), so having MVDs on `Model` in that combined table violates 4NF.

Fix (To Reach 4NF)

Split into tables, separating the independent multivalued facts:

- `Model_Colors_Available(Model, Color)`
- `Model_Styles_Available(Model, Style)`

Now:

- Adding green for Prairie = just add one row to `Model_Colors_Available`.
- No risk of forgetting a "style combination," because styles are handled independently.

Fifth Normal Form (5NF)

5NF handles the rare cases where even 4NF is not enough. It concerns tables that represent a relationship which can be decomposed into smaller relationships and recombined via joins without losing information, but where storing the combined table directly leads to anomalies.

Ice Cream Example

Brands: Frosty's, Alpine, Ice Queen. Each brand offers certain flavors:

- Frosty's: vanilla, chocolate, strawberry, mint choco chip.
- Alpine: vanilla, rum raisin.
- Ice Queen: vanilla, strawberry, mint choco chip.

People and their preferences:

- Brand preferences:

  - Jason likes: Frosty's, Alpine.
  - Suzy likes: Alpine, Ice Queen.

- Flavor preferences:
  - Jason likes: vanilla, chocolate.
  - Suzy likes: rum raisin, mint choco chip, strawberry.

From these three pieces of information:

1. Which brands offer which flavors.
2. Which people like which brands.
3. Which people like which flavors.

we can deduce which specific products each person will eat and store in a combined table: `Person_Brand_Flavor(Person, Brand, Flavor)`.

The Problem

If we maintain this big table directly, any update (e.g., Suzy now also likes Frosty's) risks forgetting some combinations that should logically exist (e.g., Frosty's + mint choco chip + Suzy). This leads to contradictions: we know she likes Frosty's and mint choco chip individually, yet the table says she does not eat that combination.

Core Idea of 5NF

If a table can be logically reconstructed by joining multiple smaller tables, and those smaller tables represent the *primitive facts*, then we should store only those smaller tables and let the DB compute the join when needed.

Fix (To Reach 5NF)

Instead of `Person_Brand_Flavor`, store:

- `Brand_Flavors(Brand, Flavor)`
- `Person_Brands(Person, Brand)`
- `Person_Flavors(Person, Flavor)`

When the combined picture is needed, join them using SQL.

Informal 5NF Rule

For a table already in 4NF:

- Ask: can it be seen as the result of joining simpler tables that share its key?
- If yes in a non-trivial way, it may not be in 5NF.
- If no such non-trivial decomposition exists, then it is in 5NF.

Global Summary & Design Heuristics

To comply with a higher normal form, a table must also satisfy all lower normal forms.

Summary of Rules

1NF

- No row-order semantics.
- No mixed data types in a column.
- Table must have a primary key.
- No repeating groups.

2NF

Each non-key attribute depends on the *entire* primary key (no part-key dependencies).

3NF

Each non-key attribute depends on the key, the whole key, and nothing but the key (no transitive dependencies on other non-key attributes).
BCNF: Strengthen to "each attribute" rather than "each non-key attribute."

4NF

The only multivalued dependencies allowed are those on a key.

5NF

The table should not be logically expressible as a non-trivial join of other tables with the same key (no further lossless decomposition based solely on join dependencies).

Practical Advice

"Every attribute depends on the key, the whole key, and nothing but the key."

If you keep this in mind, you will, in almost all real-world situations, end up with fully normalized tables (3NF/BCNF).

These are the conceptual "safety levels" of relational design; as you normalize up the ladder

$$\text{1NF} \rightarrow \text{2NF} \rightarrow \text{3NF/BCNF} \rightarrow \text{4NF} \rightarrow \text{5NF}$$

you systematically eliminate redundancy and logical contradictions, at the cost of more tables and more joins when querying.